Esercitazione – Cannone 2D, farfalle e cuori

Dalle geometrie condivise a una scena animata e interattiva

OpenGL 3.3 · GLFW · GLM · Dear ImGui · C++

---

1 Obiettivo

Realizzare una scena bidimensionale in cui un **cannone** ruota verso il **mirino del mouse** e lancia proiettili contro **farfalle e cuori**. In una prima fase i bersagli sono fermi; successivamente diventano numerosi, si muovono autonomamente e reagiscono alle collisioni. Un bersaglio colpito **scompare e non viene rigenerato**; torna disponibile soltanto dopo il ripristino della partita.

L'esercitazione consolida le trasformazioni di modellazione e introduce: conversione delle coordinate del mouse, direzioni e velocita', tempo tra frame, geometrie condivise, istanze, gestione dello stato e collision detection.

2 Materiale di partenza e comportamento iniziale

Utilizzare il `template studenti` fornito , gia' dotato delle classi `Scene`, `Geometry`, `Renderer`, `Shader`, `ShapeGenerator`, `Gui` e `Input`. Il programma iniziale deve mostrare:

- base del cannone da posizionare in $C = (0, -78)$ e canna da posizionare in verticale;
- un mirino GLFW gia' funzionante;
- una farfalla fissa in $(-38, 45)$ e un cuore fisso in $(38, 45)$;
- proiezione ortografica, interfaccia ImGui e rendering gia' predisposti.

I file delle classi e gli shader sono forniti: le attivita' riguardano soprattutto i blocchi `TODO` di `main.cpp`. Non e' necessario ricreare VAO, VBO o shader.

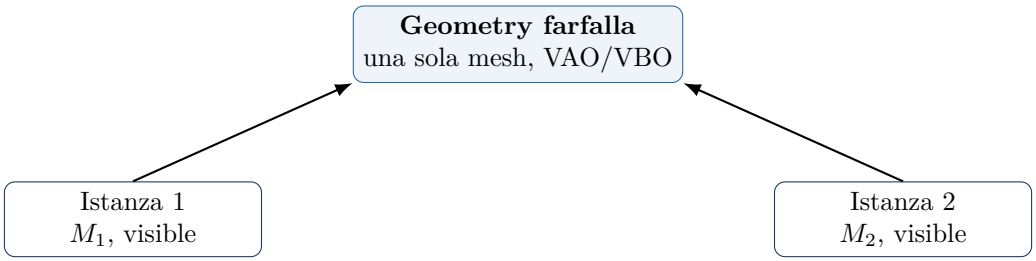
3 Prima di programmare: capire la nuova Scene

La `Scene` mantiene *separati* i dati geometrici e le loro occorrenze nel mondo:

```
std::vector<std::unique_ptr<Geometry>> m_geometries;
std::vector<GeometryInstance> m_instances;
```

Una `Geometry` contiene la mesh locale e gestisce le risorse GPU. Una `GeometryInstance` identifica una geometria condivisa e conserva una trasformazione e uno stato di visibilita':

```
struct GeometryInstance {
    std::size_t geometryIndex = 0;
    glm::mat4 modelMatrix{1.0f};
    bool visible = true;
};
```



La stessa geometria puo' essere disegnata piu' volte con matrici diverse:

$$P_{\text{clip}}^{(i)} = P_{\text{ortho}}\, M_i\, P_{\text{local}}.$$

Attivita' di osservazione. Individuare in `main.cpp` le chiamate a `scene.addGeometry(...)` e `scene.addInstance(...)`. Spiegare perche' 15 farfalle non richiedono 15 copie dei VBO. Individuare anche le istanze della base e della canna: **il cannone fa parte della Scene**.

4 Lo stato dinamico: la struttura Target

Ogni bersaglio possiede un proprio stato logico, distinto dalla rappresentazione grafica:

```
enum class TargetType { Butterfly, Heart };
struct Target {
    TargetType type;
    std::size_t instanceIndex;
    glm::vec2 position;
    glm::vec2 direction;
    float directionTimer = 0.0f;
    bool alive = true;
};
```

Campo	Significato
`type`	Distingue farfalla e cuore (scala e raggio di collisione).
`instanceIndex`	Individua la `GeometryInstance` nella `Scene`.
`position`	Centro del bersaglio in coordinate del mondo.
`direction`	Versore della direzione di movimento.
`directionTimer`	Secondi mancanti al prossimo cambio casuale di direzione.
`alive`	Stato logico: falso dopo un colpo, fino al Reset.

Distinzione chiave: `Target` descrive il comportamento; `GeometryInstance` descrive trasformazione e visibilita'; `Geometry` descrive la forma. Quando cambia `position`, occorre sincronizzare la `modelMatrix` della sua istanza.

Parte I – Richieste dell'esercitazione

5 TODO 1 – Orientare il cannone verso il mirino

Richiesta. Fare in modo che la canna segua il mouse, mantenendo fisso il proprio punto di attacco alla base.

1. Recuperare la posizione del mouse in coordinate del mondo mediante `cursorToWorld(...)`.
2. Calcolare il vettore dalla base al mirino $\mathbf{a} = \mathbf{p}_{\text{mouse}} - C$.
3. Normalizzare $\mathbf{a}$, gestendo il caso in cui il mirino coincida con la base.
4. Calcolare $\theta = \text{atan2}(a_y, a_x)$ e costruire la Model Matrix della canna.

$$M_{\text{canna}} = T(C)R_z(\theta).$$

6 TODO 2 – Attivare un proiettile

Richiesta. Quando l'utente spara, attivare un proiettile nella posizione della bocca del cannone e assegnargli una velocita' iniziale.

$$\mathbf{p}_{\text{bocca}} = C + L\,\mathbf{d}, \qquad \mathbf{v} = s\,\mathbf{d},$$

dove L e' la lunghezza della canna, $\mathbf{d}$ la direzione normalizzata e s la velocita' selezionata nella GUI.

1. Esaminare il pool `std::vector<Projectile>` predisposto nel codice.
2. Nella funzione `fireProjectile(...)` cercare il primo slot con `active == false`.
3. Impostare `active = true`, `position = origin`, `velocity = direction * speed`.
4. Interrompere la ricerca: uno sparo attiva un solo proiettile. Se il pool e' pieno, non crearne altri.

7 TODO 3 – Animare i proiettili con deltaTime

Richiesta. Aggiornare i proiettili attivi con moto rettilineo uniforme, indipendente dal numero di frame al secondo.

$$\boxed{\mathbf{p}_{\text{nuova}} = \mathbf{p}_{\text{vecchia}} + \mathbf{v}\,\Delta t}$$

1. Calcolare Δt come differenza fra due letture successive di `glfwGetTime()` (nel template e' gia' predisposto).
2. In `updateProjectiles(...)` aggiornare `p.position` soltanto per i proiettili attivi.
3. Disattivare un proiettile quando esce dalla regione visibile.
4. Impostare la visibilita' della corrispondente istanza in funzione di `p.active`; se attiva, aggiornarne la Model Matrix con una traslazione.

```
// Relazione fondamentale da applicare nel TODO:
p.position += p.velocity * dt;
```

8 TODO 4A – Visualizzare piu' bersagli fissi

Richiesta. Eliminare il limite provvisorio di un solo cuore e una sola farfalla e utilizzare i conteggi della GUI. Non modificare il numero massimo di istanze preallocate.

1. In `updateTargets(...)` distinguere farfalle e cuori con due contatori.
2. Abilitare soltanto le prime `activeButterflies` farfalle e i primi `activeHearts` cuori.
3. Per ogni bersaglio, impostare `instance.visible = enabled && b.alive`.
4. Ricostruire la Model Matrix delle istanze visibili: $M_i = T(\mathbf{p}_i)S_i$.

```
// Esempio della logica di visibilita':
instance.visible = enabled && b.alive;
```

9 TODO 4B – Muovere i bersagli e farli rimbalzare

Richiesta. Introdurre il movimento autonomo dei bersagli ancora vivi e visibili.

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + s\,\mathbf{d}(t)\,\Delta t, \qquad \|\mathbf{d}(t)\| = 1.$$

1. Per ogni bersaglio abilitato e vivo, aggiornare `b.position` usando `b.direction`, `speed` e `dt`.
2. Gestire il rimbalzo sui bordi della regione destinata ai bersagli: su un bordo verticale invertire la componente x della direzione, su uno orizzontale la componente y.
3. Aggiornare `instance.modelMatrix` *dopo* aver aggiornato la posizione.
4. Diminuire `directionTimer` di `dt`; quando scade, ruotare la direzione di un piccolo angolo casuale e assegnare un nuovo intervallo al timer.

10 TODO 5A – Collisione proiettile–bersaglio

Richiesta. Rilevare quando un proiettile colpisce un bersaglio vivo e visibile, approssimando le figure con cerchi.

$$\boxed{\|\mathbf{p} - \mathbf{b}\| \le r_p + r_b} \quad \Longleftrightarrow \quad (\mathbf{p} - \mathbf{b}) \cdot (\mathbf{p} - \mathbf{b}) \le (r_p + r_b)^2.$$

1. Per ogni proiettile attivo, visitare i bersagli vivi e visibili.
2. Scegliere il raggio del bersaglio in base a `TargetType`.
3. Confrontare il quadrato della distanza tra i centri con il quadrato della somma dei raggi.
4. In caso di impatto, disattivare il proiettile, segnare il bersaglio come eliminato, nasconderne l'istanza e incrementare il punteggio.

```
// Comportamento richiesto in caso di collisione:
p.active = false;
b.alive = false;
scene.instance(b.instanceIndex).visible = false;
++score;
```

Non chiamare `respawnTarget()`: il bersaglio colpito non deve ricomparire. Interrompere il ciclo interno dopo il primo impatto del proiettile.

11 TODO 5B – Evitare sovrapposizioni tra bersagli (approfondimento)

Richiesta. Far reagire farfalle e cuori quando entrano in contatto, senza eliminarli. Controllare tutte le coppie $i < j$ di bersagli vivi e visibili.

$$\mathbf{d} = \mathbf{p}_B - \mathbf{p}_A, \quad \delta = r_A + r_B - \|\mathbf{d}\|.$$

Se $\delta > 0$, i cerchi si sovrappongono. Per $\|\mathbf{d}\| > 0$, calcolare la normale $\mathbf{n} = \mathbf{d}/\|\mathbf{d}\|$ e separare i centri:

$$\mathbf{p}_A \leftarrow \mathbf{p}_A - \tfrac{\delta}{2}\mathbf{n}, \qquad \mathbf{p}_B \leftarrow \mathbf{p}_B + \tfrac{\delta}{2}\mathbf{n}.$$

1. Gestire separatamente il caso dei centri coincidenti, scegliendo una normale valida.
2. Modificare le direzioni quando i bersagli si stanno avvicinando, usando la componente relativa lungo la normale.
3. Aggiornare le Model Matrix dopo la correzione delle posizioni.
4. Se necessario, ripetere la correzione piu' volte per ridurre sovrapposizioni residue in scene affollate.

12 Ripristino della partita (Reset)

Il Reset deve ripristinare la *configurazione iniziale* e non generare nuove posizioni casuali, almeno nella versione di base dell'esercitazione.

1. Azzerare `score` e `fireCooldown`.
2. Per ogni `Target`, ripristinare `alive = true` e la posizione iniziale. Per la prima farfalla e il primo cuore: rispettivamente $(-38, 45)$ e $(38, 45)$.
3. Ripristinare, se necessario, anche direzione e timer per una ripartenza riproducibile.
4. Aggiornare la Model Matrix di ciascuna istanza in modo coerente con la posizione logica.
5. Disattivare e nascondere tutti i proiettili; lasciare a `updateTargets()` la selezione finale delle istanze abilitate dalla GUI.

Attenzione. La posizione `b.position` e la traslazione contenuta nella `modelMatrix` devono coincidere: altrimenti il bersaglio visibile potrebbe non coincidere con il centro usato per la collisione.

13 Ordine corretto delle operazioni nel render loop

Il ciclo principale deve seguire un ordine coerente:

1. Lettura eventi e calcolo di Δt; acquisizione dei valori della GUI.
2. Eventuale Reset; conversione del mouse e aggiornamento della Model Matrix della canna.
3. Gestione dello sparo; aggiornamento delle posizioni dei bersagli.
4. Eventuale risoluzione delle collisioni tra bersagli.
5. Aggiornamento dei proiettili e collisioni proiettile–bersaglio.

6. Disegno della `Scene` e dell'interfaccia ImGui; scambio dei buffer.

Il `Renderer` attraversa `scene.instances()`, salta quelle non visibili e disegna la geometria condivisa usando la matrice specifica dell'istanza. Durante l'animazione **non occorre riscrivere i VBO**.

Parte II – Suggerimenti e verifiche per ogni TODO

13.1 TODO 1 – Orientare il cannone verso il mirino

Come procedere. Recuperare la posizione del mouse in coordinate del mondo e sottrarre la posizione della base. Il vettore risultante deve essere normalizzato prima di calcolare la bocca della canna. Se il vettore ha lunghezza nulla, utilizzare temporaneamente la direzione $(0, 1)$. La canna e' modellata lungo $+x$, quindi usare atan2(d_y, d_x). Costruire la trasformazione $T(C)R_z(\theta)$, non $R_z(\theta)T(C)$.

Suggerimento. La mesh della canna e' costruita lungo l'asse locale $+x$, con origine nel perno. Per questo l'angolo e' misurato rispetto a $+x$. In GLM l'ordine TR significa: prima rotazione locale, poi traslazione nel mondo.

Verifica. Con il mirino sopra la base, la canna e' verticale (90°); spostando il mouse a destra e sinistra ruota senza staccarsi dalla base.

13.2 TODO 2 – Attivare un proiettile

Come procedere. Esaminare uno slot alla volta con un ciclo sul vettore `projectiles`. Saltare quelli attivi; al primo inattivo assegnare origine e velocita' e terminare con `return`. Controllare che la posizione di origine sia la bocca del cannone e non `BASE_POSITION`.

Suggerimento. Il proiettile possiede gia' una istanza grafica nella Scene. Non aggiungere una nuova Geometry a ogni sparo. Osservare inoltre il controllo sull'input ImGui e sul cooldown.

Verifica. Con un clic o con la barra spaziatrice viene attivato uno slot del pool. Il proiettile nasce all'estremita' della canna, non al centro della finestra.

13.3 TODO 3 – Animare i proiettili con deltaTime

Come procedere. Separare aggiornamento logico e rendering: prima cambiare `p.position`, poi impostare `instance.visible` e `instance.modelMatrix`. L'aggiornamento e' in unita' di mondo al secondo, quindi deve includere `dt`. Non creare nuovi VBO durante il movimento.

Verifica. Il proiettile procede lungo la direzione di sparo; cambiando il frame rate non deve cambiare sensibilmente la sua velocita'. Gli slot inattivi non vengono disegnati.

13.4 TODO 4A – Visualizzare piu' bersagli fissi

Come procedere. Partire dalla versione con una farfalla e un cuore. Rimuovere il limite provvisorio `std::min(...,1)` mantenendo i due contatori separati. Distinguere `enabled` (richiesta GUI) da `alive` (stato del gioco). Per ciascuna istanza visibile usare la traslazione coerente con `b.position`.

Suggerimento. Il contatore `butterfliesSeen` indica quante farfalle sono gia' state incontrate nel ciclo; non e' il numero di farfalle vive. Un bersaglio colpito non deve essere sostituito automaticamente dal successivo.

Verifica. Gli slider mostrano piu' farfalle e cuori, ma i VBO sono ancora condivisi. Una farfalla colpita rimane invisibile anche se si modificano gli slider.

13.5 TODO 4B – Muovere i bersagli e farli rimbalzare

Come procedere. Implementare prima soltanto il moto rettilineo, poi i rimbalzi, infine i cambi casuali. Sottrarre `dt` dal timer e modificare la direzione solo quando scade. Dopo ogni correzione di posizione aggiornare la Model Matrix; non modificare i vertici della mesh.

Suggerimento. Per iniziare usare i limiti per i centri $-88 \leq x \leq 88$, $0 \leq y \leq 88$; successivamente considerare anche il raggio del bersaglio. Evitare di cambiare direzione casualmente a ogni frame.

Verifica. Farfalle e cuori si muovono indipendentemente, restano nell'area assegnata e non ricompaiono dopo essere stati colpiti.

13.6 TODO 5A – Collisione proiettile–bersaglio

Come procedere. Iniziare con un solo bersaglio fermo e confrontare la distanza tra i centri con la somma dei raggi. Poi passare al prodotto scalare per evitare la radice quadrata. Dopo il colpo impostare `p.active=false`, `b.alive=false`, `instance.visible=false`; non richiamare `respawnTarget()`.

Verifica. Sparando alla farfalla scompare soltanto la farfalla; sparando al cuore scompare soltanto il cuore. Il punteggio aumenta una sola volta per impatto.

13.7 TODO 5B – Evitare sovrapposizioni tra bersagli (approfondimento)

Come procedere. Esaminare ogni coppia una sola volta con $i < j$. Prima limitarsi a riconoscere la sovrapposizione; poi separare i centri lungo la normale. Gestire il caso di distanza nulla. Solo in un secondo momento aggiungere la risposta semplificata sulle direzioni. Le istanze invisibili e i bersagli eliminati vanno ignorati.

Suggerimento. La risposta agli urti puo' essere semplificata: non e' richiesta una simulazione fisica rigorosa. La correzione deve riguardare soltanto bersagli vivi e visibili.

Verifica. Due farfalle, due cuori o una farfalla e un cuore non devono attraversarsi visibilmente; nessuno dei due viene eliminato per questo contatto.

13.8 Ripristino della partita (Reset)

Come procedere. Evitare `respawnTarget()` nel Reset della versione base. Conservare o ricostruire le posizioni iniziali; dopo averle ripristinate, aggiornare immediatamente le Model Matrix. Ripristinare `alive=true`, disattivare i proiettili e lasciare alla GUI la selezione delle istanze da mostrare.

Verifica. Dopo Reset il cuore e la farfalla ricompaiono nelle posizioni iniziali e possono essere nuovamente colpiti.

13.9 Ordine corretto delle operazioni nel render loop

Come procedere. Controllare che la Model Matrix venga aggiornata prima di `drawScene()`. Separare il calcolo della posizione, la risposta alle collisioni e il disegno: il Renderer non deve conoscere le regole del gioco.